UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Branch: MCA (Data Science)	**Semester:** 2
Student Name: Drishti Bhatia	**UID:** 25MCD10008
Subject Name: Technical Training - I Lab	**Subject Code**: 25CAP-652
Section/Group: 25MCD- KAR1	**Date of Performance:** 31-03-2026

Experiment No.: 7

1. **Aim of the practical:**
   Implementation of joins in PostgreSQL (inner join, left join, right join, self join and cross join)

2. **Tools Used:** PostgreSQL

3. **Objectives of the practical:**
   Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)

4. **Theory:**
   Joins in SQL are used to combine data from two or more tables based on a common column. They are an essential feature of relational databases that allow users to retrieve related data stored in different tables in a single query.

   There are different types of joins used in SQL:

   1. INNER JOIN:
   INNER JOIN returns only those records that have matching values in both tables. If there is no match, the rows are not included in the result.

   2. LEFT JOIN (LEFT OUTER JOIN):
   LEFT JOIN returns all records from the left table and the matching records from the right table. If there is no match, NULL values are returned for the right table columns.

   3. RIGHT JOIN (RIGHT OUTER JOIN):
   RIGHT JOIN returns all records from the right table and the matching records from the left table. If there is no match, NULL values are returned for the left table columns.

   4. FULL JOIN (FULL OUTER JOIN):
   FULL JOIN returns all records from both tables. It includes matching rows as well as non-matching rows from both sides, filling NULL where no match exists.

   5. CROSS JOIN:
   CROSS JOIN returns all possible combinations of rows from both tables. It is also known as the Cartesian product.

   6. SELF JOIN:
   SELF JOIN is used to join a table with itself. It is useful when comparing rows within the same table.

Joins help in reducing data redundancy, improving data retrieval efficiency, and performing complex queries by combining related information from multiple tables.

5. **Experiment / Practical Steps:**

Creating tables and inserting values into them

```
-- STUDENTS TABLE
CREATE TABLE studentt (
   student_id INT PRIMARY KEY,
   student_name VARCHAR(50),
   dept_id INT
);

-- COURSES TABLE
CREATE TABLE courses (
   course_id INT PRIMARY KEY,
   course_name VARCHAR(50)
);

-- ENROLLMENTS TABLE (relation table)
CREATE TABLE enrollments (
   student_id INT,
   course_id INT,
   PRIMARY KEY (student_id, course_id)
);

-- DEPARTMENTS TABLE
CREATE TABLE departmentss (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50)
);

-- STUDENTS
INSERT INTO studentt VALUES
(101, 'Aman', 1),
(102, 'Riya', 2),
(103, 'Rahul', 3),
(104, 'Sneha', 4),
(105, 'Arjun', 1);

-- COURSES
INSERT INTO courses VALUES
(201, 'Database'),
(202, 'Cloud'),
(203, 'AI'),
(204, 'Networking');

-- ENROLLMENTS
INSERT INTO enrollments VALUES
```

```
(101, 201),
(101, 202),
(102, 203),
(103, 201),
(104, 204);

-- DEPARTMENTS
INSERT INTO departmentss VALUES
(1, 'Oracle'),
(2, 'SAP'),
(3, 'IBM'),
(4, 'Microsoft');
```

1: Write queries to list students with their enrolled courses (INNER JOIN).

Query:

```
SELECT s.student_id, s.student_name, c.course_name
FROM studentt s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id;
```

Output:

	student_id integer	student_name character varying (50)	course_name character varying (50)
1	101	Aman	Database
2	101	Aman	Cloud
3	102	Riya	AI
4	103	Rahul	Database
5	104	Sneha	Networking

2: Find students not enrolled in any course (LEFT JOIN).

Query:

```
SELECT s.student_id, s.student_name
FROM studentt s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

Output:

	student_id [PK] integer	student_name character varying (50)
1	105	Arjun

3: Display all courses with or without enrolled students (RIGHT JOIN).

Query:

```
SELECT s.student_name, c.course_name
FROM enrollments e
RIGHT JOIN courses c ON e.course_id = c.course_id
LEFT JOIN studentt s ON e.student_id = s.student_id;
```

Output:

	student_name character varying (50)	course_name character varying (50)
1	Aman	Database
2	Aman	Cloud
3	Riya	AI
4	Rahul	Database
5	Sneha	Networking

4: Show students with department info using SELF JOIN or multiple joins.

Query:

```
SELECT s.student_id, s.student_name, d.dept_name
FROM studentt s
INNER JOIN departmentss d ON s.dept_id = d.dept_id;
```

Output:

	student_id integer	student_name character varying (50)	dept_name character varying (50)
1	101	Aman	Oracle
2	102	Riya	SAP
3	103	Rahul	IBM
4	104	Sneha	Microsoft
5	105	Arjun	Oracle

5: Display all possible student-course combinations (CROSS JOIN). (Oracle, SAP, IBM, Microsoft)

Query:

```
SELECT s.student_name, c.course_name
FROM studentt s
CROSS JOIN courses c;
```

Output:

	student_name character varying (50)	course_name character varying (50)
1	Aman	Database
2	Riya	Database
3	Rahul	Database
4	Sneha	Database
5	Arjun	Database
6	Aman	Cloud
7	Riya	Cloud
8	Rahul	Cloud
9	Sneha	Cloud
10	Arjun	Cloud
11	Aman	AI
12	Riya	AI
13	Rahul	AI
14	Sneha	AI
15	Arjun	AI
16	Aman	Networking
17	Riya	Networking
18	Rahul	Networking
19	Sneha	Networking
20	Arjun	Networking

6. Learning Outcomes:

- Understood the concept of joins in SQL and their importance in relational databases.
- Applied different types of joins such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and CROSS JOIN in queries.
- Analyzed how data from multiple tables can be combined using common attributes.
- Implemented SQL queries to retrieve meaningful and structured data from related tables.